

IEOR 6617 Project: Hybrid Vision Transformer

Yujie Yang

December 15th 2025

1 Introduction

1.1 background

With the rapid rise of large language models, the Transformer architecture has become one of the most influential and widely adopted models nowadays. Besides natural language processing tasks, Transformers have also demonstrated strong performance in computer vision after the introduction of the Vision Transformer (ViT) in 2021.

Attention is the core component of Transformer models. However, standard self-attention is computationally expensive, as it requires large-scale matrix multiplications whose time and memory complexity grow quadratically with the input sequence length. To address this limitation, Performer technique provides a approach to approximating softmax attention with linear complexity. this project is designed to compare the performance and efficiency of different Vision Transformer model which some of its attention were replaced by performer. This project aims to compare both the performance and efficiency (speed) of different Vision Transformer variants in which standard attention layers are partially replaced by Performer-based attention. We evaluate these hybrid models based on image classification tasks and analyze the trade-off between accuracy, training time, and inference efficiency.

1.2 Objective

This project aims to compare both the performance and efficiency (speed) of different Vision Transformer variants in which standard attention layers are partially replaced by Performer-based attention. We evaluate these hybrid models based on image classification tasks and analyze the trade-off between accuracy, training time, and inference efficiency. Performer-ReLU and Performer FAVOR+ are used in the experiments.

2 Model

2.1 Regular Transformer

By introducing the attention mechanism, the Transformer effectively addresses key limitations of earlier models, including the difficulty of modeling long-range dependencies in sequences and the lack of efficient parallelization[3].

A standard Transformer architecture mainly consists of four components: input-encoder-decoder-output. The input layer includes token embeddings combined with positional encoding. The encoder is composed of stacked layers, each containing multi-head self-attention, a position-wise feed-forward neural network (FFN), residual connections, and

layer normalization. The decoder has a similar structure, but additionally includes masked multi-head self-attention and encoder–decoder attention to control information flow during autoregressive generation. The output layer produces predicted token probabilities by applying a linear projection followed by a softmax function to the decoder outputs.

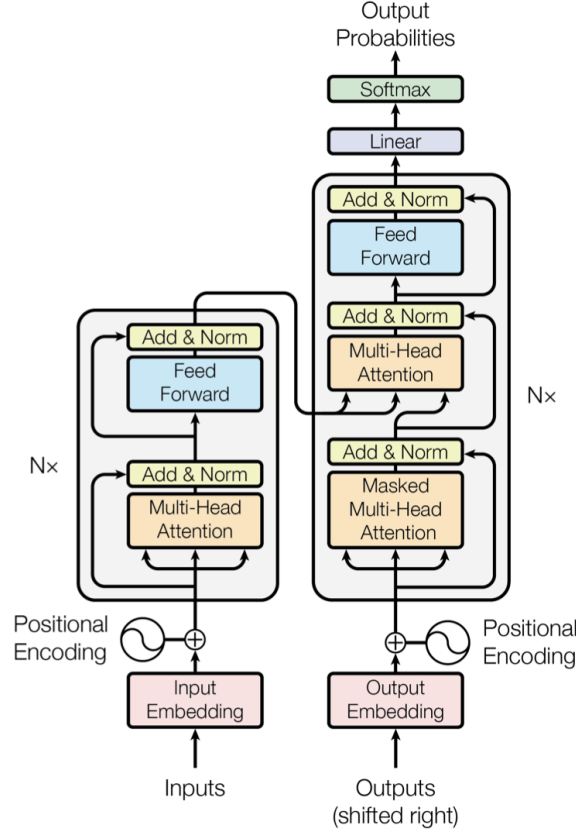


Figure 1: The Transformer architecture [3]

2.2 From Transformer to Vision Transformer

Introduced in 2021, Vision Transformer, a variant of transformer, mainly includes consists of an input embedding module, an encoder, and an output head[2].

2.3 Vision Transformer Architecture[2]

2.3.1 Patch Partitioning (Patching)

Given an input image $x \in \mathbb{R}^{H \times W \times C}$, ViT splits it into N non-overlapping patches of size $P \times P$ (each patch has $P^2 \cdot C$ values). The number of patches is

$$N = \frac{HW}{P^2}. \quad (1)$$

After flattening each patch into a 1-D vector, we obtain the patch matrix

$$X_{\text{patches}} \in \mathbb{R}^{N \times (P^2 C)}. \quad (2)$$

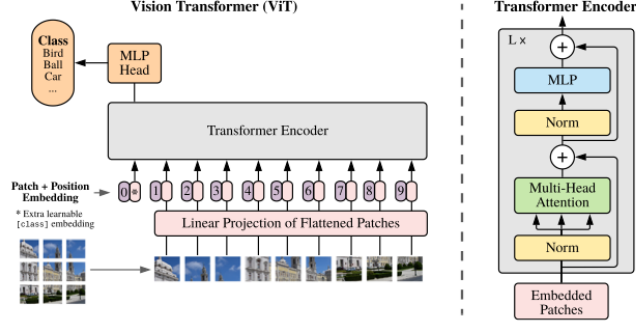


Figure 2: The Vision Transformer architecture [2]

2.3.2 Positional Encoding

Since patch embeddings do not inherently contain spatial order information, ViT incorporates positional encoding to preserve the spatial relationships among image patches.

$$Z' = Z + P, \quad (3)$$

where $P \in \mathbb{R}^{N \times D}$.

The positional encoding is defined using sinusoidal functions:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/D}}\right), \quad \text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/D}}\right), \quad (4)$$

which ensures bounded values and allows the model to capture relative positional information.

2.3.3 Transformer Encoder

The Encoder is the main computational block of the Vision Transformer. It consists of multiple stacked layers, each composed of a Multi-Head Self-Attention (MHSA) module followed by a feed-forward network (FFN), enabling the model to capture global dependencies among all image patches.

Multi-Head Self-Attention (MHSA) Given the input embeddings with positional information $Z' \in \mathbb{R}^{N \times D}$, the query, key, and value matrices are computed as:

$$Q = Z'W_Q, \quad K = Z'W_K, \quad V = Z'W_V, \quad (5)$$

where $W_Q, W_K, W_V \in \mathbb{R}^{D \times D}$.

The attention matrix is then computed using scaled dot-product attention:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right), \quad (6)$$

and the attention output is obtained by weighting the value matrix:

$$Z_{\text{attn}} = AV. \quad (7)$$

For multi-head attention, the attention outputs from all heads are concatenated and projected:

$$\text{MSA}(Z') = \text{concat} \left(Z_{\text{attn}}^{(1)}, Z_{\text{attn}}^{(2)}, \dots, Z_{\text{attn}}^{(h)} \right) W_O, \quad (8)$$

where $W_O \in \mathbb{R}^{(hD) \times D}$.

Feed-Forward Network and Residual Connections Each encoder layer applies layer normalization and residual connections to stabilize training. The output of the attention block is:

$$Z_1 = \text{LayerNorm} \left(Z' + \text{MSA}(Z') \right). \quad (9)$$

A position-wise feed-forward network is then applied:

$$\text{FFN}(Z_1) = \text{ReLU}(Z_1 W_1 + b_1) W_2 + b_2, \quad (10)$$

where $W_1 \in \mathbb{R}^{D \times D_{\text{hidden}}}$ and $W_2 \in \mathbb{R}^{D_{\text{hidden}} \times D}$.

Finally, another residual connection and layer normalization are applied:

$$Z_2 = \text{LayerNorm} \left(Z_1 + \text{FFN}(Z_1) \right). \quad (11)$$

This encoder block is repeated for multiple layers.

2.3.4 CLS Token and Classification Head

The classification token ([CLS]) is a special learnable token prepended to the patch embedding sequence. It is intended to aggregate global information from all image patches and serve as the final representation of the input image for classification tasks.

2.3.5 Output Layer

For prediction, the final representation of the input image is obtained from the [CLS] token output of the Transformer encoder. This representation is passed through a task-specific output layer, typically implemented as a feed-forward (linear) layer.

For classification tasks, the output layer applies a linear projection followed by a softmax function to produce class probabilities:

$$\hat{y} = \text{softmax}(Z_{[\text{CLS}]} W_c + b_c), \quad (12)$$

where W_c and b_c denote the classifier weights and bias, respectively. The resulting probability distribution $\hat{y}$ is used to generate the final prediction.

2.4 From Attention to Performer

2.4.1 Complexity of Standard Self-Attention

Assume the input to a self-attention layer be a sequence of N tokens, each with embedding dimension D . The input embedding matrix is denoted as

$$X \in \mathbb{R}^{N \times D}. \quad (13)$$

The query, key, and value matrices are obtained via linear projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (14)$$

where

$$W_Q, W_K, W_V \in \mathbb{R}^{D \times D}, \quad (15)$$

and thus

$$Q, K, V \in \mathbb{R}^{N \times D}. \quad (16)$$

The attention score matrix is computed as:

$$S = QK^\top, \quad (17)$$

where

$$Q \in \mathbb{R}^{N \times D}, \quad K^\top \in \mathbb{R}^{D \times N}, \quad (18)$$

resulting in

$$S \in \mathbb{R}^{N \times N}. \quad (19)$$

After scaling and applying the softmax function, the attention output is obtained by:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right), \quad Z = AV, \quad (20)$$

where

$$A \in \mathbb{R}^{N \times N}, \quad V \in \mathbb{R}^{N \times D}, \quad Z \in \mathbb{R}^{N \times D}. \quad (21)$$

Computational Cost The dominant cost arises from computing the matrix multiplication $QK^\top$. Therefore, the computation complexity of standard self-attention is $\mathcal{O}(N^2D)$, which scales quadratically with the sequence length N .

2.4.2 Complexity of Performer

Performer replaces the softmax attention with a kernel-based approximation using a feature map $\phi(\cdot)$. Let m denote the number of random features (kernel features). After computing $Q, K, V \in \mathbb{R}^{N \times D}$ as in standard attention, Performer applies a feature mapping to queries and keys:

$$\phi(Q) \in \mathbb{R}^{N \times m}, \quad \phi(K) \in \mathbb{R}^{N \times m}. \quad (22)$$

Instead of explicitly forming the $N \times N$ attention matrix, Performer computes attention as

$$Z \approx \frac{\phi(Q) (\phi(K)^\top V)}{\phi(Q) (\phi(K)^\top \mathbf{1})}, \quad (23)$$

where $\mathbf{1} \in \mathbb{R}^{N \times 1}$ is an all-ones vector and the division is applied row-wise (i.e., via normalization of each token).

Computational Cost Performer avoids constructing the $N \times N$ matrix. The dominant multiplications are:

$$\phi(K)^\top V : \mathcal{O}(NmD), \quad \phi(Q) (\phi(K)^\top V) : \mathcal{O}(NmD). \quad (24)$$

Thus, the overall complexity scales as

$$\mathcal{O}(NmD), \quad (25)$$

which is linear in the sequence length N when $m \ll N$. The memory usage is reduced from $\mathcal{O}(N^2)$ to $\mathcal{O}(Nm)$.

2.5 Performer Variants

2.5.1 FAVOR+

FAVOR+ (Fast Attention Via positive Orthogonal Random features) is an unbiased and numerically stable Performer variant designed to approximate the softmax attention kernel. The key idea is to rewrite the softmax kernel as an expectation over positive random features.

Specifically, the softmax kernel can be expressed as:

$$\exp(q^\top k) = \mathbb{E}_{\omega \sim \mathcal{N}(0, I)} \left[\exp(\omega^\top q - \tfrac{1}{2}\|q\|^2) \exp(\omega^\top k - \tfrac{1}{2}\|k\|^2) \right]. \quad (26)$$

[1]

FAVOR+ defines the feature map:

$$\phi(x) = \frac{1}{\sqrt{m}} \exp \left(\omega x - \tfrac{1}{2}\|x\|^2 \right), \quad (27)$$

where $\omega \in \mathbb{R}^{m \times D}$ is sampled from a Gaussian distribution and orthogonalized to reduce variance.

2.5.2 Performer-ReLU

Performer-ReLU is a generalized attention variant that replaces the softmax kernel with a ReLU-based feature map. It defines the kernel as:

$$K(q, k) = \phi(q)^\top \phi(k), \quad (28)$$

with the feature mapping:

$$\phi(x) = \text{ReLU}(x). \quad (29)$$

This formulation ensures non-negativity of attention weights and allows the attention computation to remain linear in the sequence length using the same Performer framework.

3 Experiments

3.1 Dataset

3.1.1 MNIST

MNIST is a benchmark dataset for handwritten digit recognition, consisting of grayscale images of digits from 0 to 9. It contains 60,000 training images and 10,000 test images, each with a resolution of 28×28 pixels and a single channel. Due to its simplicity and standardized setup, MNIST is commonly used to evaluate and compare image classification models.

In experiments, MNIST images were first resized from 28×28 to 32×32 to ensure compatibility with patch-based Vision Transformer models. The images were then converted to tensors and normalized using the standard MNIST mean and standard deviation. The training set was shuffled and loaded in mini-batches, while the test set was loaded without shuffling. The images were normalized using the standard mean and standard deviation of the MNIST dataset.

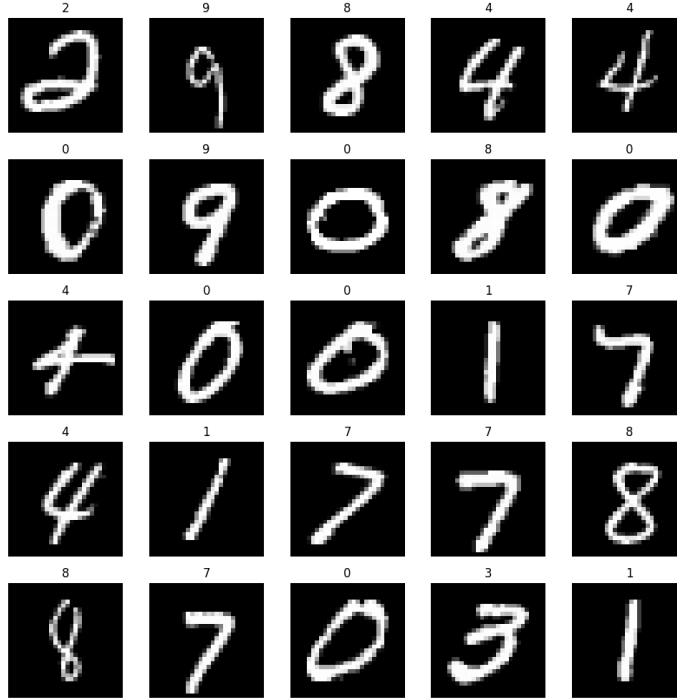


Figure 3: Examples of MNIST handwritten digit images.

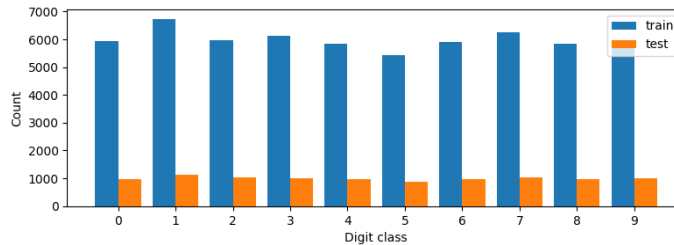


Figure 4: Class distribution of the MNIST dataset.

3.1.2 CIFAR-10

CIFAR-10 is a benchmark dataset for natural image classification, consisting of color images from 10 object categories such as airplanes, automobiles, animals, and ships. The dataset contains 50,000 training images and 10,000 test images, each with a resolution of 32×32 pixels and three color channels. Compared to MNIST, CIFAR-10 presents a more challenging classification task due to its higher visual complexity and richer semantic content.

In experiments, CIFAR-10 images were used in their original 32×32 RGB format without resizing. The images were converted to tensors and normalized using the standard channel-wise mean and standard deviation of the CIFAR-10 dataset. The training set was shuffled and loaded in mini-batches, while the test set was loaded without shuffling. This preprocessing ensures compatibility with patch-based Vision Transformer models while preserving the original image resolution.

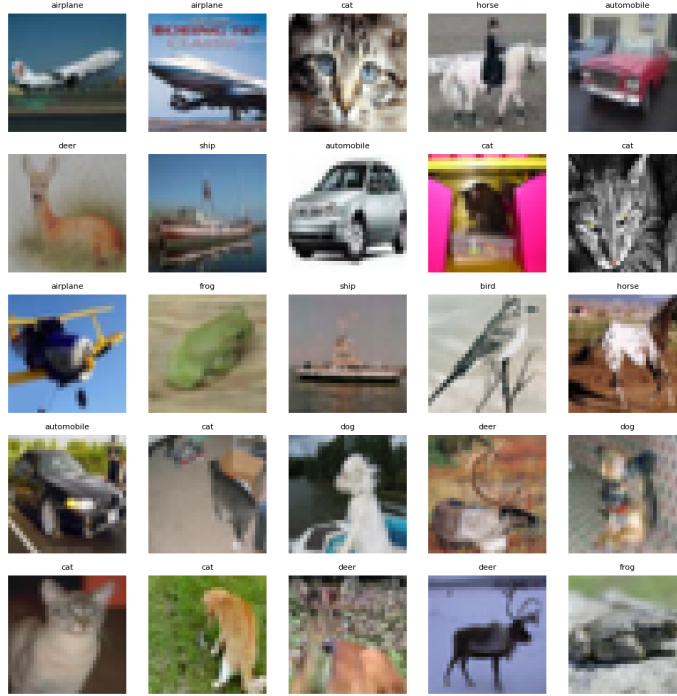


Figure 5: Examples of CIFAR10 images.

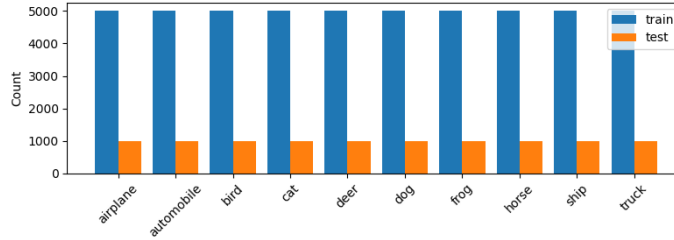


Figure 6: Class distribution of the CIFAR10.

3.2 Experimental Models

In this project, the standard Vision Transformer (ViT) is used as the baseline model. Two Performer variants, FAVOR+ and Performer-ReLU, are incorporated into hybrid Transformer architectures, where Performer attention replaces the regular self-attention modules in ViT.

For intertwined hybrid models, two layer patterns are evaluated: **prprpr** and **rprprp**, where **p** denotes Performer attention and **r** denotes regular self-attention. For *stacked* hybrid models, two patterns are considered: **ppprrr** and **rrrppp**. All models are evaluated on both the MNIST and CIFAR-10 datasets.

The influence of the number of kernel features m is also investigated. The parameter m is specific to the FAVOR+ Performer and controls the Monte Carlo approximation accuracy of the softmax attention kernel. In contrast, Performer-ReLU employs a deterministic feature mapping; therefore, the parameter m is not applicable and is not varied for Performer-ReLU models.

3.3 Training Setup

3.3.1 Hardware

All experiments were completed on Google Colab with the "change runtime type" option: "T4 GPU".

3.3.2 Hyperparameters

In our experiments, we adopted a lightweight Vision Transformer configuration to balance model capacity and computational efficiency. The model uses 4 attention heads with a hidden dimension of 64 and consists of 6 Transformer encoder layers. Each image is divided into 4×4 patches, and the feed-forward network dimension is set to 256.

For training, the model is optimized for 10 epochs using a batch size of 128. The learning rate is set to 3×10^{-4} with a weight decay of 0.05. A dropout rate of 0.1 is applied throughout the network to mitigate overfitting.

4 Results

Choice of the kernel feature dimension m . In our implementation, the model dimension is set to $d_{\text{model}} = 64$ with $h = 4$ attention heads, resulting in a per-head dimension of

$$d_{\text{head}} = \frac{d_{\text{model}}}{h} = 16.$$

For the FAVOR+ Performer, the hyperparameter m controls the number of random features used to approximate the softmax attention kernel. To study the effect of m relative to the intrinsic dimensionality of each attention head, we evaluate $m \in \{4, 8, 16, 32\}$, which corresponds to values smaller than, equal to, and larger than d_{head} . This choice allows us to examine whether increasing the number of random features beyond the head dimension leads to improved approximation accuracy and downstream classification performance, or merely increases computational cost without significant gains.

Baseline Vision Transformer. The baseline Vision Transformer achieves 96.96% test accuracy on MNIST and 62.79% on CIFAR-10. The total training times are 229.02s (MNIST) and 178.48s (CIFAR-10). Inference over 10,000 test samples takes 2.30s and 2.25s, corresponding to 0.230ms and 0.225ms per sample, respectively.

4.1 Dataset: MNIST

Performer-ReLU on MNIST

Model Configuration	Test Accuracy (%)	Training Time (s)
ReLU (RRRPPP)	97.00	270.87
ReLU (RPRPRP)	97.28	246.66
ReLU (PRPRPR)	97.06	255.68
ReLU (PPPRRR)	97.31	245.36

FAVOR+ Performer on MNIST (PPPRRR)

m	Test Accuracy (%)	Training Time (s)
4	97.23	282.19
8	97.70	283.07
16	96.97	283.44
32	97.17	292.60

FAVOR+ Performer on MNIST (PRPRPR)

m	Test Accuracy (%)	Training Time (s)
4	96.71	315.75
8	97.53	315.79
16	96.70	316.64
32	97.58	322.33

FAVOR+ Performer on MNIST (RPRPRP)

m	Test Accuracy (%)	Training Time (s)
4	95.87	292.20
8	96.76	289.19
16	96.78	289.87
32	97.03	300.65

FAVOR+ Performer on MNIST (RRRPPP)

m	Test Accuracy (%)	Training Time (s)
4	96.80	288.87
8	96.85	284.92
16	96.57	296.83
32	97.25	306.13

4.2 Dataset: CIFAR-10

Performer-ReLU on CIFAR-10

Model Configuration	Test Accuracy (%)	Training Time (s)
ReLU (RRRPPP)	62.92	204.16
ReLU (RPRPRP)	63.19	226.98
ReLU (PRPRPR)	62.16	198.22
ReLU (PPRRRR)	62.33	191.05

FAVOR+ Performer on CIFAR-10 (PPRRRR)

m	Test Accuracy (%)	Training Time (s)
4	60.71	238.46
8	61.25	235.48
16	60.66	235.91
32	60.55	242.94

FAVOR+ Performer on CIFAR-10 (PRPRPR)

m	Test Accuracy (%)	Training Time (s)
4	61.94	227.38
8	61.01	232.13
16	61.74	232.58
32	60.55	238.62

FAVOR+ Performer on CIFAR-10 (RPRPRP)

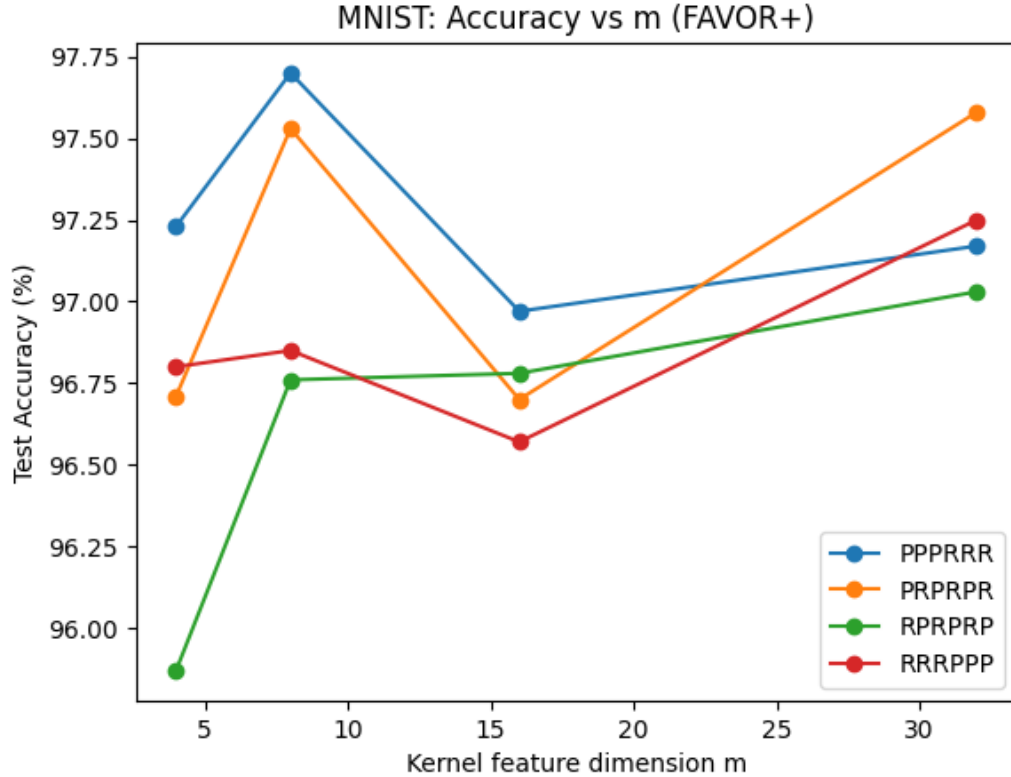
m	Test Accuracy (%)	Training Time (s)
4	61.15	246.24
8	61.40	247.21
16	62.05	249.20
32	61.88	259.31

FAVOR+ Performer on CIFAR-10 (RRRPPP)

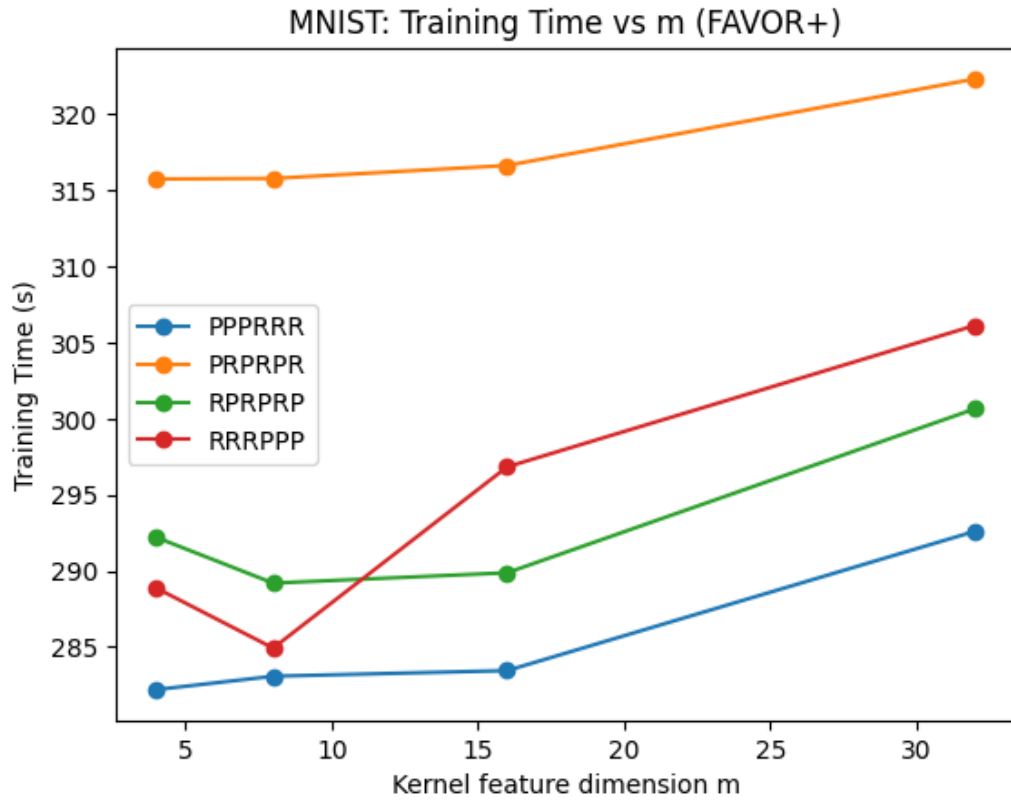
m	Test Accuracy (%)	Training Time (s)
4	62.32	266.28
8	61.91	266.89
16	62.57	266.99
32	62.23	271.27

4.3 Visualization of m

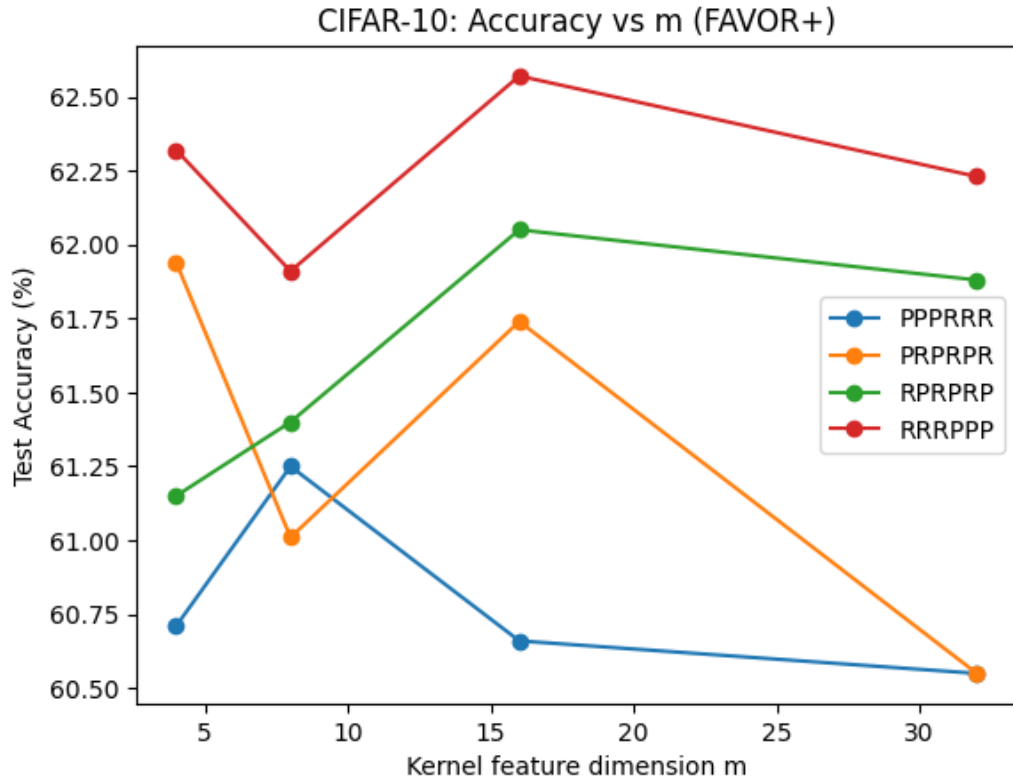
MNIST: Accuracy vs. Kernel Feature Dimension m



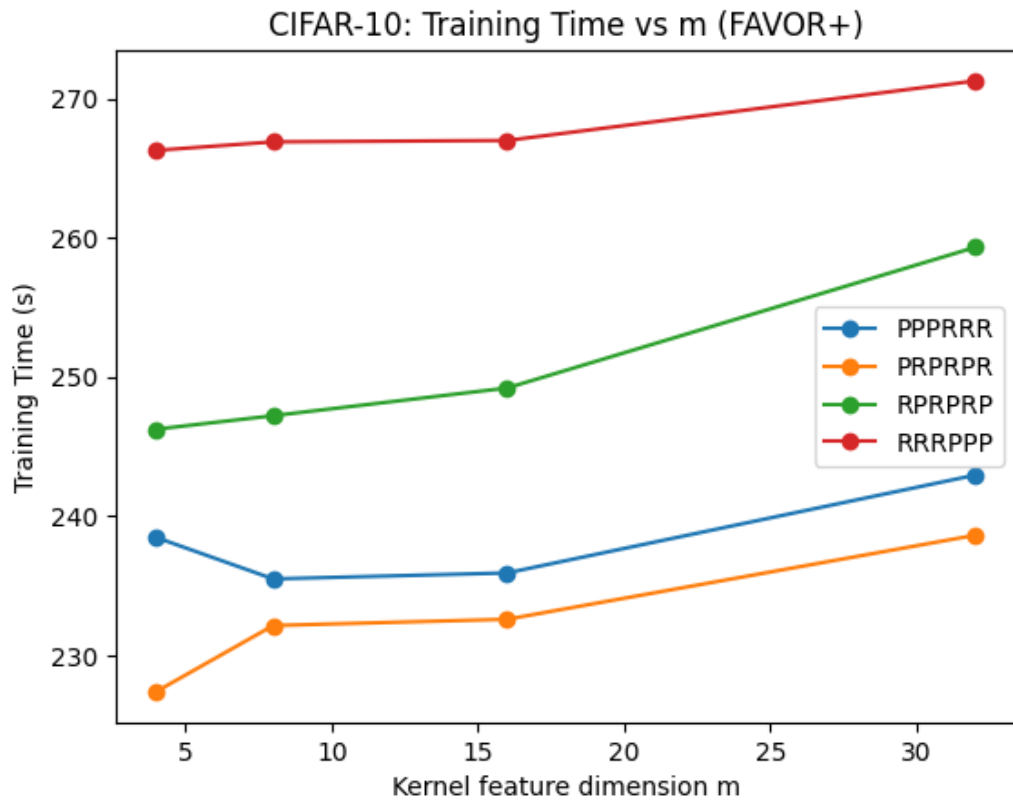
MNIST: Training Time vs. Kernel Feature Dimension m



CIFAR-10: Accuracy vs. Kernel Feature Dimension m



CIFAR-10: Training Time vs. Kernel Feature Dimension m



5 Discussion

5.1 Overall Performance Comparison

Across both MNIST and CIFAR-10 datasets, the hybrid Transformer models consistently achieve performance comparable to or slightly better than the baseline Vision Transformer. On MNIST, all hybrid configurations reach test accuracies above 96.5%, with several FAVOR+ and Performer-ReLU variants slightly surpassing the baseline accuracy. On CIFAR-10, although the overall accuracy is lower due to the increased task complexity, hybrid models remain competitive with the baseline ViT, and no configuration shows a severe performance degradation.

These results indicate that replacing standard attention layers with Performer-based approximations does not significantly harm classification accuracy, even when multiple Performer layers are stacked or interleaved with regular attention layers.

5.2 Effect of Kernel Feature Dimension m

For FAVOR+ Performer models, the kernel feature dimension m plays an important role in controlling the trade-off between approximation quality and computational cost. In our experiments, we evaluate $m \in \{4, 8, 16, 32\}$, covering values below, equal to, and above the per-head attention dimension ($d_{\text{head}} = 16$).

On MNIST, the test accuracy does not increase monotonically with m . Most configurations achieve their best or near-best performance when m is close to 16, while further increasing m provides no consistent accuracy improvement. A similar trend is observed on CIFAR-10, where accuracy fluctuates across different values of m and does not consistently benefit from larger kernel feature dimensions.

These observations suggest that once the kernel feature dimension is comparable to the intrinsic dimensionality of each attention head, increasing m mainly increases computational cost without yielding substantial gains in accuracy.

5.3 Layer Placement Strategy

The placement strategy of Performer and regular attention layers has a noticeable impact on both training efficiency and performance stability. For MNIST, configurations such as PP-PRRR and PRPRPR achieve strong performance with relatively moderate training times, indicating that placing Performer layers in earlier stages of the network can be effective. On CIFAR-10, the differences among placement strategies become more pronounced, with some configurations exhibiting better accuracy–efficiency trade-offs than others.

Overall, no single placement strategy universally dominates across all settings. However, hybrid models with a balanced or front-loaded Performer placement tend to offer a favorable compromise between accuracy and computational efficiency, making them attractive alternatives to the fully regular Vision Transformer.

6 Conclusions

In this project, I systematically investigated hybrid Vision Transformer models that combine regular self-attention and Performer-based attention mechanisms on MNIST and CIFAR-10 datasets.

Regarding the effect of the kernel feature dimension m , our experiments show that classification accuracy does not increase monotonically as m becomes larger.

Although, in theory, a larger m provides a more accurate approximation of the softmax attention by covering a richer feature space, this trend is not consistently observed in practice. We attribute this behavior to the use of a lightweight model configuration in our experiments, where limited model capacity and training duration constrain the potential benefits of increasing m , leading to diminishing returns in classification performance.

Across both datasets, most FAVOR+ Performer configurations achieve their best or near-best performance when m is comparable to the per-head attention dimension ($d_{\text{head}} = 16$). Increasing m beyond this range mainly increases computational cost, while providing little or no consistent improvement in accuracy. This observation suggests that a moderate number of random features is sufficient to approximate the softmax attention kernel in practice.

With respect to the placement strategy of regular and Performer layers, no single configuration is universally optimal across all settings. However, hybrid models that place Performer layers in the early or middle stages of the network, such as PPPRRR and PRPRPR, consistently exhibit a favorable balance between accuracy and training efficiency on both MNIST and CIFAR-10.

In contrast, fully stacked or heavily back-loaded Performer configurations do not offer clear advantages in accuracy and often incur higher computational cost.

Overall, results indicate that hybrid Transformer architectures can effectively reduce the reliance on full softmax attention without sacrificing performance, provided that the kernel feature dimension and layer placement are chosen appropriately.

References

- [1] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021.
- [2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Appendix

A Statement of Team Member Contributions

This project was completed by Yujie Yang (team of one person).

B Experimental Codes

All experiments were conducted using Google Colab.

The experimental codes are publicly available at the following GitHub repository:

<https://github.com/yyj20251029/IEOR-6617-Project.git>